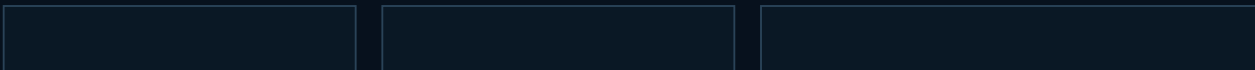


PART 5 · SOLUTION PRESENTATION

A practical walkthrough of the implementation approach, architecture, evidence, decisions, and lessons.



```
[ ] [ ] [ ] presentation opening
```

```
# outcome
$ git log --oneline -3
b8324a9 Kubernetes evidence
f5d58e1 Docker Hub deploy
7a51dac validation-first build
```

EXECUTIVE SNAPSHOT

A layered solution with proof at every boundary

01 · FOUNDATION

Linux

Commands, system health, process and file operations documented with evidence.

02 · PACKAGING

Docker

Separate API/UI images with Compose for repeatable local execution.

03 · DELIVERY

CI/CD

GitHub Actions verifies, builds, tests, publishes, and reports status.

04 · RESILIENCE

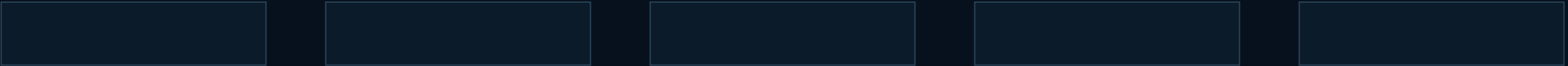
Kubernetes HA

Replicas, probes, ingress, HPA, PDB, and runtime verification.

exam areas

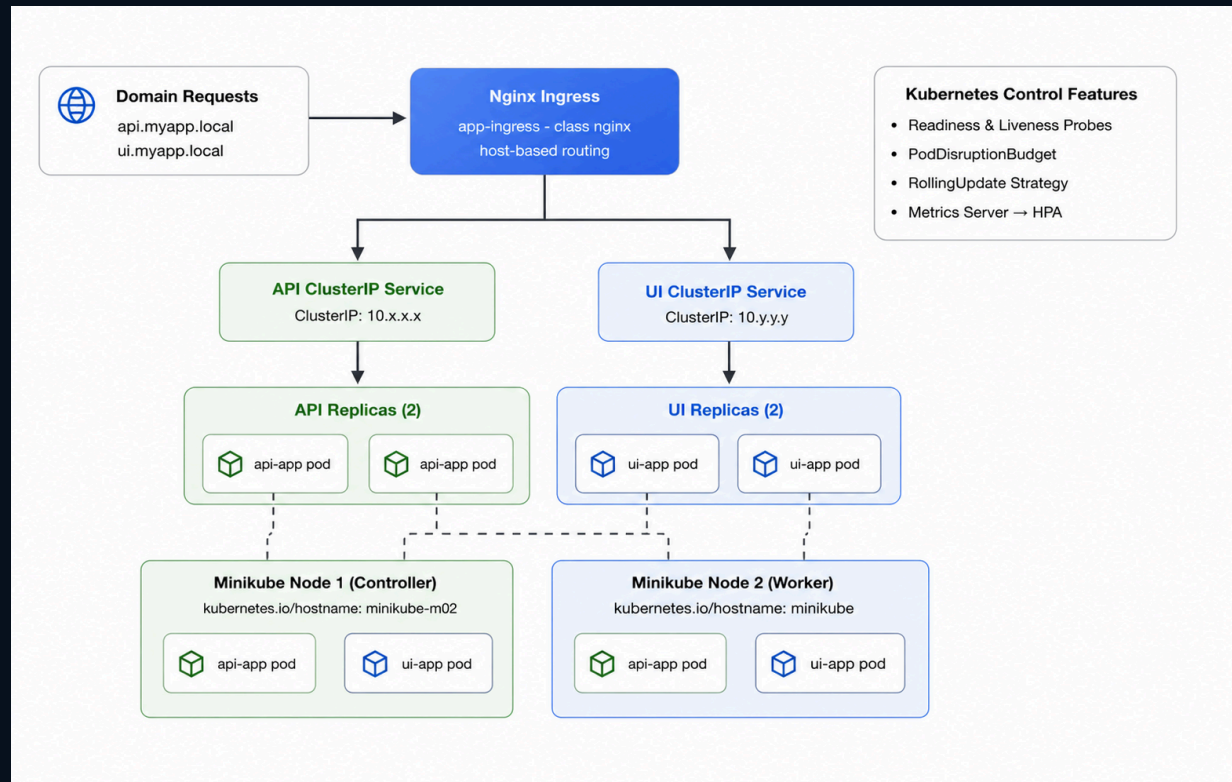
distribution requests

node-failure HTTP 200



ARCHITECTURE

One application path, three execution environments



LOCAL FEEDBACK LOOP

Docker Compose

API, UI, and MySQL share an isolated bridge network.

DELIVERY BOUNDARY

GitHub Actions ? Docker Hub

Tests run before publishing; SHA tags provide traceability.

RUNTIME BOUNDARY

Kubernetes / Minikube

Replicas, services, ingress, probes, HPA, and PDB.

PART 1 · LINUX FOUNDATIONS

I treated the shell as the first production interface

```
draiimon@Aloof:~$ nano devops-exam/scripts/system_health.sh
draiimon@Aloof:~$ chmod +x devops-exam/scripts/system_health.sh
draiimon@Aloof:~$ bash devops-exam/scripts/system_health.sh

=====
SYSTEM HEALTH REPORT
=====

[1] DATE & TIME
Thu Aug 6 11:52:56 CST 2026

[2] UPTIME
11:52:56 up 14 min, 1 user, load average: 0.02, 0.23, 0.15

[3] MEMORY USAGE
total      used      free      shared  buff/cache  available
Mem:    1.9Gi    584Mi    96Mi     3.1Mi     1.4Gi     1.4Gi
Swap:      0B       0B       0B

[4] DISK SPACE
Filesystem      Size  Used Avail Use% Mounted on
none            987M    0 987M   0% /usr/lib/modules/5.15.167.4-microsoft-standard-WSL2
none            987M  4.8K 987M   1% /mnt/wsl
drivers         465G  398G   75G  84% /usr/lib/wsl/drivers
/dev/sdb        1007G   2.2G 954G   1% /
none            987M   88K 987M   1% /mnt/wslg
none            987M    0 987M   0% /usr/lib/wsl/lib
rootfs          983M   2.4M 981M   1% /init
none            987M  496K 986M   1% /run
none            987M    0 987M   0% /run/lock
none            987M    0 987M   0% /run/shm
tmpfs           4.0M    0 4.0M   0% /sys/fs/cgroup
none            987M   76K 987M   1% /mnt/wslg/versions.txt
none            987M   76K 987M   1% /mnt/wslg/doc
C:\             465G  398G   75G  84% /mnt/c
D:\             50M   16M   35M  32% /mnt/d
H:\            932G  845G   87G  91% /mnt/h
tmpfs           198M   16K 198M   1% /run/user/1000

[5] TOP 5 PROCESSES BY CPU
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         53  0.6  0.7 66820 16092 ?        S<s  11:38   0:05 /usr/lib/systemd/systemd-journald
root         1   0.2  0.6 22108 13464 ?        Ss   11:38   0:02 /sbin/init
root      1156  0.1  3.0 2014724 61048 ?        Ssl  11:42   0:01 /usr/bin/containerd
root      1267  0.0  5.0 2130632 101780 ?        Ssl  11:42   0:00 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd
.sock
message+  145  0.0  0.2  9772  5300 ?        Ss   11:38   0:00 @dbus-daemon -system --address=systemd: --nofork --nopidfile --s
ystemd-activation --syslog-only

[6] CPU INFO
4
0.02 0.23 0.15 1/191 2795

=====
END OF REPORT
=====
draiimon@Aloof:~$ |
```

[OK] Inspect system information, packages, users, permissions, and networking.

[OK] Analyze processes and logs with standard Unix text-processing tools.

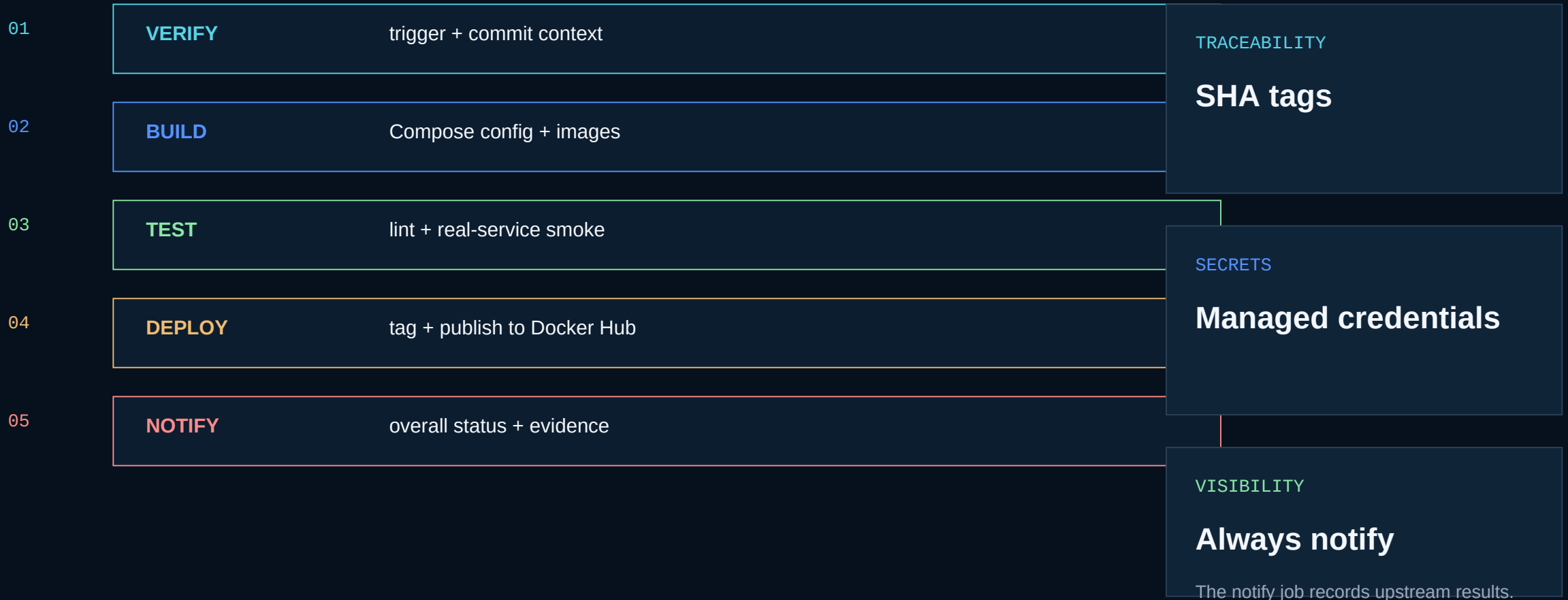
[OK] Automate health, backup, and log analysis as executable Bash scripts.

[OK] Document commands with screenshots and explanations.

```
[ ] [ ] [ ] part1-linux/
```


PART 3 · CI/CD PIPELINE

The pipeline is a quality gate, not just a deployment button



PART 3 · EVIDENCE

The delivery path is backed by observable results

[illegible]

CONFIRMED

[OK] Build + test passed on GitHub-hosted runners.

[OK]

[OK] Success notification captured in the evidence trail.

PART 4 · HIGH AVAILABILITY DESIGN

Availability is designed into the workload, not added after failure

REPLICATION

Two API replicas

maxUnavailable: 0 and topology spread ask the scheduler to separate pods.

TRAFFIC

Stable service path

ClusterIP + Ingress provide a stable route while EndpointSlice tracks Ready backends.

FEEDBACK

Health + scale

Readiness gates traffic, liveness restarts, and HPA responds to load.

```
[ ] [ ] [ ] Kubernetes object model
```

DESIGN PRINCIPLE

PART 4 · EVIDENCE

The runtime checks the story the manifests promised

```

Context: minikube [m]      <=> all          <ctrl-d> Delete          <syn_YAML>
Cluster: minikube          <=> devops-exam <=> Describe
User: minikube             <=> default      <=> Edit
WSR Rev: v0.81.0           <=>             <=> Help
WSR Rev: v1.35.1           <=>             <=> Jump Owner
CPU: 1%                    <=>             <=> Warp To Namespace
MEM: 10%

horizontalpodautoscalers(devops-exam)[2]
NAME      REFERENCE                TARGETS          MINPODS   MAXPODS   REPLICAS   AGE
-----
ui-hpa    Deployment/ui-app         cpu: 8%  mem: memory: 81%/80%      2         4         2         25m

```

RUNTIME VERIFIER · --PREFLIGHT

[OK] Rollout completes successfully.

[OK] At least two Ready pods with pod IPs.

```
[OK] All Ready pods use the expected image.
```

[OK] Every Ready pod IP is in the EndpointSlice.

[OK] Ingress path preserves /instance.

[OK] 40 requests reached both identities within 2:1.

```
[OK] Captured node test: 20/20 HTTP 200.
```

PART 5 · THINKING AND TRADE-OFFS

The strongest decisions made failure visible

CHALLENGE ? RESOLUTION

Fresh CI runners

Challenge: state from one job does not exist in the next.

Resolution: each job checks out code and declares its own setup.

CHALLENGE -> RESOLUTION

Mutable staging tags

Challenge: rollout can look healthy with stale or mixed image state.

Resolution: SHA tags plus EXPECTED_API_IMAGE preflight.

CHALLENGE -> RESOLUTION

Ingress path behavior

Challenge: rewrite-target turned /instance into a 404.

Resolution: remove stale annotation and verify path handling.

TRADE-OFF

Minikube vs managed Kubernetes

Minikube is reproducible and exam-friendly, but not production control-plane durability. Next: repeat manifests in managed staging.

LIVE WALKTHROUGH RUNBOOK

A calm demo is a prepared demo

```
[ ] [ ] [ ] demo order · do not improvise
```

```
# 1. repository state
$ git status --short --branch
$ git log --oneline --decorate -3

# 2. local service contract
$ docker compose -f part2-docker/docker-compose.yml config --quiet
? compose configuration valid

# 3. HA readiness gate
$ EXPECTED_API_IMAGE=draiimon112/devops-api:staging \
  ./part4-ha/verify-runtime-evidence.sh --preflight
```

RULE 01

Show the boundary

RULE 02

Explain before output

State what the command should prove before running it.

RULE 03

Have an evidence fallback

CLOSING STATEMENT

Reliable delivery is a chain of verified boundaries

Reliable delivery is a chain of verified boundaries.

Linux makes state observable. Docker makes services portable. CI/CD makes delivery repeatable. Kubernetes makes availability testable.

WITH MORE TIME

[OK] Security gate with image scanning.

[OK] Immutable digests in deployment manifests.

[OK] A tested rollback procedure.

[OK] Production-grade observability.

Thank you · Questions welcome

SAY

Good day. I will present the solution as a delivery story: Linux and container foundations, an automated path to a registry, and a two-replica Kubernetes workload with runtime evidence.

WALK-THROUGH

Open the repository README and point to the four parts. Start with the outcome, not a code dump.

IF ASKED

Scope: Linux basics, Docker containerization, CI/CD, and high availability deployment.

SAY

My approach was incremental and evidence-driven. Each layer produces an artifact the next layer can consume: scripts and docs, images, a pipeline, then Kubernetes manifests and runtime verification.

WALK-THROUGH

```
tree -L 2  
sed -n '1,220p' README.md  
git status --short --branch
```

IF ASKED

This order reduces debugging scope: if the container is not healthy, Kubernetes is not the place to debug the application.

SAY

The key decision is separation of concerns. Compose provides local feedback, GitHub Actions provides repeatable delivery, and Kubernetes provides availability controls.

WALK-THROUGH

```
Show: part2-docker/docker-compose.yml, .github/workflows/deploy.yml, part4-ha/k8s/
```

IF ASKED

Separate services allow independent scaling, clearer health boundaries, and a smaller blast radius.

SAY

Before containers or Kubernetes, I established operational habits: inspect the system, understand processes and networking, manage files safely, and automate repeated checks.

WALK-THROUGH

```
bash part1-linux/system_health.sh
bash part1-linux/backup.sh
bash part1-linux/log_analysis.sh
```

IF ASKED

The scripts turn ad-hoc diagnosis into a repeatable baseline.

SAY

I used separate multi-stage images for the API and UI because they have different runtimes, dependencies, ports, and scaling behavior. Compose adds the MySQL dependency.

WALK-THROUGH

```
Open the two Dockerfiles and docker-compose.yml. Run docker compose ... config --quiet, then ps.
```

IF ASKED

If the local contract is unstable, the CI smoke test should fail before publishing.

SAY

The pipeline is deliberately modular: verify, build, test, deploy, and notify. Only after the test stage passes does Deploy publish Docker Hub tags.

WALK-THROUGH

Open `.github/workflows/deploy.yml` and point to the jobs in dependency order.

IF ASKED

Fresh runners make hidden local state visible, so each job must declare its dependencies and readiness checks.

SAY

These screenshots show successful build, test, publish, and notification proof. Optional image scanning and a live rollback exercise are follow-up work, not overclaimed as complete.

WALK-THROUGH

Show CI success, success email, and Docker Hub tags. Then show the current workflow file.

IF ASKED I did not intentionally break a working pipeline just to create failure-email evidence.

SAY

Kubernetes provides the primitives the requirement needs: replicas, scheduling constraints, probes, stable Services, Ingress, autoscaling, and disruption protection.

WALK-THROUGH

```
Open api-deployment.yaml, api-service.yaml, hpa.yaml, poddisruptionbudget.yaml, and verify-runtime-evidence.sh.
```

IF ASKED

Minikube is a reproducible local exam environment; the manifests can be repeated in managed Kubernetes.

SAY

The important detail is the preflight. I do not count requests just because rollout says complete. I first verify pods, images, EndpointSlice membership, ingress behavior, and /instance.

WALK-THROUGH

```
EXPECTED_API_IMAGE=draiimon112/devops-api:staging ./part4-ha/verify-runtime-evidence.sh --preflight  
./part4-ha/verify-runtime-evidence.sh  
./part4-ha/verify-runtime-evidence.sh --node-failure
```

IF ASKED

A Ready pod not present in the EndpointSlice is not receiving traffic, so pod status alone can mislead.

SAY

The most useful challenges were boundary problems: fresh CI runners, mutable image tags, and ingress path behavior. I resolved them with explicit readiness, SHA traceability, and preflight checks.

WALK-THROUGH

Use the slide as the reflection section. Clearly separate completed work, documented strategy, and next improvements.

IF ASKED

Next improvements: image scanning, immutable deployment digests, tested rollback, and managed staging.

SAY

I will keep the demo deterministic: repository state, Compose validation, workflow shape, then Kubernetes preflight or captured evidence. I will not improvise if the cluster is unavailable.

WALK-THROUGH

```
git status --short --branch  
git log --oneline --decorate -3  
docker compose -f part2-docker/docker-compose.yml config --quiet
```

IF ASKED

If a command fails, stop at that boundary and explain whether it is environment readiness or application behavior.

SAY

My main takeaway is that reliability is not one tool. It is a chain of verified boundaries. I can show the repository, explain the choices, and reproduce the checks.

WALK-THROUGH

Finish with: I am happy to walk through any file or command in more detail.

IF ASKED

Never infer runtime correctness from one green signal; verify the boundary that carries traffic.